DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Modelado de autómatas en VHDL**	**CÓDIGO:** **AINP3**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Individual. Lectura dirigida.	TIPO:	No presencial	DURACIÓN:	60 minutos

CALENDARIO:	Antes de la sesión presencial S10	REQUISITOS:	

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

Introducción

En actividades anteriores ha visto cómo modelar algunos circuitos secuenciales sencillos en VHDL. En esta actividad aprenderá una metodología general de modelado de autómatas en dicho lenguaje. Se presentará una característica nueva de VHDL, la definición de tipos de usuario, como ayuda para el proceso de modelado.

Estructura general de un autómata

Como ya sabe, un autómata genérico puede descomponerse en tres partes (véase la Figura 1):

1. La **memoria de estado**, que almacena el estado del autómata en cada momento. Es el elemento que distingue a un circuito secuencial de uno combinacional.

2. La **lógica de cálculo del siguiente estado**, un circuito combinacional que genera, a partir del estado actual y de las entradas, el estado futuro.

3. La **lógica de cálculo de las salidas**, otro circuito combinacional que genera, a partir del estado actual y (opcionalmente) de las entradas, las salidas del autómata.

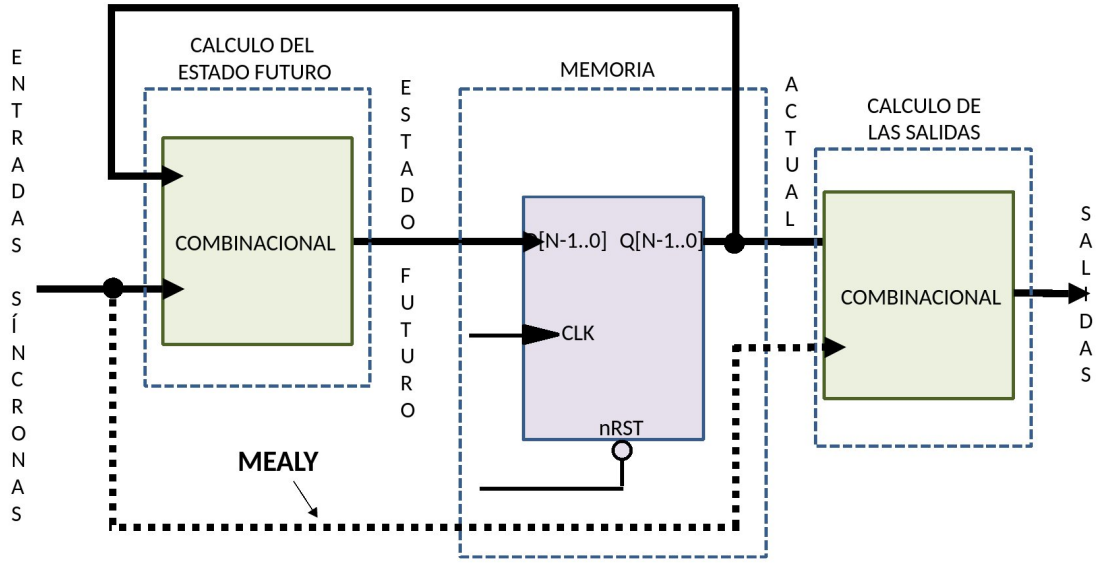


Figura 1: Estructura general de un autómata

Recordará también que los autómatas se clasifican, en función de si sus salidas dependen del estado actual y de las entradas o bien únicamente del estado actual, en autómatas de **Mealy** y **Moore**, respectivamente. Puede distinguirse además un caso particular en los autómatas de Moore cuando la salida es directamente el estado del autómata, en cuyo caso no existe la lógica de cálculo de las salidas.

Metodología de diseño de autómatas con VHDL

En la asignatura *Electrónica II* aprendió una metodología para el diseño de autómatas que constaba de los siguientes pasos:

1. Identificación y codificación de entradas y salidas
2. Diseño del diagrama de estados
3. Codificación de estados
4. Extracción de las tablas de transiciones y salidas
5. Diseño e implementación de la lógica de cálculo de estado futuro y salidas
6. Verificación del funcionamiento

Cuando se utiliza un flujo de diseño basado en puertas lógicas, como aprendió en *Electrónica II*, las herramientas de simulación sólo permiten verificar la corrección funcional del autómata (es decir, comprobar que el funcionamiento del autómata responde a la especificación inicial) tras finalizar todo el proceso de diseño.

El uso de VHDL permite adelantar esta verificación al final del segundo paso, creando un modelo simulable del diagrama de estados, y facilita enormemente el resto del proceso mediante el uso de herramientas de síntesis automática como *Quartus Prime*. La metodología de diseño de autómatas con VHDL consta de los siguientes pasos:

1. Identificación y codificación de entradas y salidas
2. Diseño del diagrama de estados
3. Codificación del modelo VHDL
4. Codificación del banco de test y verificación del funcionamiento
5. Síntesis automática

A continuación se presenta una guía de estilo para el modelado de autómatas en VHDL. Después se presentan ejemplos de aplicación de esta metodología para autómatas de Moore y de Mealy.

Modelado de autómatas en VHDL

En general, los autómatas pueden modelarse en VHDL utilizando dos procesos:

1. Un proceso que modela la memoria de estado y la lógica de cálculo del estado siguiente a partir de la información de estado y de las entradas del autómata.

   Este proceso es sensible únicamente a la señal de reloj y a las entradas asíncronas de inicialización.

2. Un proceso que modela la lógica de cálculo de las salidas del autómata a partir únicamente del estado actual (Moore) o del estado actual y las entradas (Mealy).

Este proceso es sensible al estado del autómata (Moore) o al estado y a las entradas del autómata (Mealy).

Tenga en cuenta que esta forma de modelar autómatas no es la única posible. En general existen múltiples opciones a la hora de realizar un modelo en VHDL de cualquier sistema, y esto es también aplicable al modelado de autómatas. Cada posibilidad puede presentar ventajas e inconvenientes respecto a otras.

Ejemplo 1: generador de paridad par

El primer ejemplo que vamos a utilizar es muy simple. Se trata de un generador de paridad par, que ante una secuencia de datos de un bit que se van presentando en serie (uno por ciclo de reloj) genera una salida de un bit tal que el número total de unos que contiene la secuencia de datos hasta el ciclo de reloj anterior más el bit de salida es par. Supondremos que el generador de paridad par cuenta además con una entrada de *reset* síncrono que indica el inicio de una nueva secuencia de datos a partir del siguiente ciclo de reloj.

Sigamos a continuación los pasos de la metodología que se presentó anteriormente:

1. Identificación y codificación de entradas y salidas

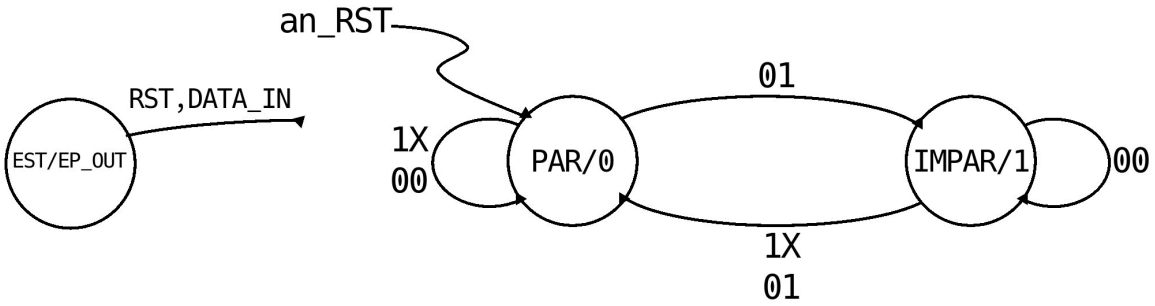
Según la especificación el autómata tiene dos entradas: la de *reset* síncrono, a la que llamaremos *RST*, y la entrada de datos de un bit, a la que llamaremos *DATA_IN*. El autómata tiene una salida de un bit a la que llamaremos *EP_OUT*. La codificación responde a la indicada en el enunciado.

Como es habitual, el autómata tendrá también una entrada de *reset* asíncrono para la inicialización. Esta entrada aparecerá en el diagrama de estados únicamente para indicar cuál es el estado inicial.

2. Diseño del diagrama de estados

El hecho de que el generador de paridad tenga en cuenta los datos en la entrada desde el inicio del funcionamiento (o desde la última inicialización) hasta el ciclo de reloj *anterior* permite implementar el autómata mediante una máquina de estados de Moore.

A continuación se muestra el diagrama de estados correspondiente al generador de paridad par:



3. Codificación del modelo VHDL

Como puede verse, en este ejemplo es inmediato asignar la codificación de estados PAR = 0 e IMPAR = 1, de modo que la salida del autómata corresponde a su estado. En casos como este la estructura general de dos procesos puede simplificarse, ya que la lógica de cálculo de las salidas a partir del estado no existe. A continuación se muestra el código que modela el autómata.

```
library ieee;
use ieee.std_logic_1164.all;

entity e_parity is
 port(an_rst, clk :in std_logic;
    rst       :in std_logic;
    data_in    :in std_logic;
    ep_out     :buffer std_logic
 );
end entity e_parity;

architecture rtl of e_parity is
begin
 process(an_rst, clk)
 begin
  if an_rst = '0' then
   ep_out <= '0';
  elsif clk'event and clk = '1' then
   if rst = '1' then
    ep_out <= '0';
   else
    case ep_out is
     when '0' =>
      if data_in = '1' then
       ep_out = '1';
      end if;
     when '1' =>
      if data_in = '1' then
       ep_out = '0';
      end if;
     when others=>
      ep_out <= 'X';
    end case;
   end if;
  end if;-- an_rst = '0'
 end process;
end architecture rtl;-- of e_parity
```

Los pasos 4 y 5 de la metodología (codificación del banco de test y simulación, y síntesis automática) se dejan, en este ejemplo y en los siguientes, como ejercicio opcional para el lector.

Fíjese en que al modelar el diagrama de estados se ha asignado previamente una codificación de estados, y este paso no aparece en la metodología de diseño de autómatas con VHDL que se ha presentado anteriormente. Esta omisión es deliberada, ya que en general la codificación de la información de estado se realiza empleando una característica del lenguaje VHDL, la definición de tipos de datos de usuario, que aprenderá en el siguiente apartado.

Codificación de la información de estado mediante tipos de datos de usuario

Una de las características del lenguaje VHDL es que permite que el usuario del lenguaje defina nuevos tipos de datos, que a partir de su definición pueden emplearse del mismo modo que los tipos intrínsecos del lenguaje. De hecho, los tipos *std_logic* y *std_logic_vector* que ha venido utilizando hasta ahora no pertenecen al lenguaje en sí, sino que se definen dentro de la librería estándar *std_logic_1164*.

Existen varias formas de definir un nuevo tipo de datos en VHDL. En este apartado presentaremos únicamente una de ellas: los tipos de datos *enumerados*. Un tipo de datos enumerado define una cantidad finita (y, normalmente, pequeña) de valores, identificados cada uno de ellos mediante un nombre. La sintaxis que se emplea para definir un tipo de datos enumerado y para emplearlo después es la siguiente:

```
Type nombre_tipo is(valor1,valor2[,...]);

signal s : nombre_tipo;
```

Siendo *nombre_tipo* el nombre que identifica al nuevo tipo de datos, y *valor1…* los nombres que identifican a cada uno de los posibles valores del nuevo tipo de datos. El ámbito de validez del tipo de datos está determinado por la ubicación de su definición; si por ejemplo se define el nuevo tipo de datos en la zona declarativa de un cuerpo de arquitectura, entonces únicamente podrá usarse dentro de ese cuerpo de arquitectura.

Ejemplo 2: detector de secuencia 011

Como segundo ejemplo diseñaremos un detector de secuencia que active su salida durante un ciclo de reloj cuando detecte en su entrada la secuencia “011”. En la entrada se presenta un nuevo dato de un bit en cada ciclo de reloj. El detector se realizará mediante una máquina de estados de Moore.

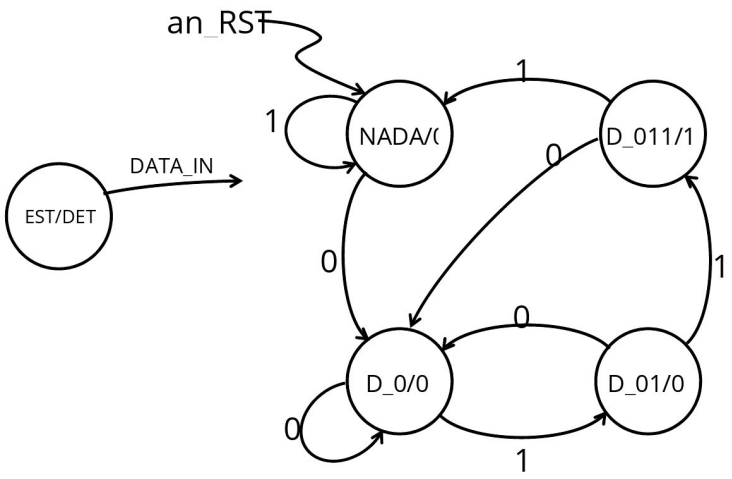
Al igual que en el primer ejemplo, seguiremos los pasos de la metodología uno por uno.

1. Identificación y codificación de entradas y salidas

Llamaremos *DATA_IN* a la entrada de datos de un bit, y *DET* a la salida, también de un bit, que se activará (‘1’) durante un ciclo de reloj cuando se detecte la secuencia “011”, permaneciendo desactivada (‘0’) el resto del tiempo.

2. Diseño del diagrama de estados

La siguiente figura muestra el diagrama de estados del detector de secuencia “011”. Examínelo detenidamente hasta asegurarse de que comprende su funcionamiento.



3. Codificación del modelo VHDL

A continuación se muestra el código del modelo VHDL del autómata. Fíjese en que hay dos procesos, uno declarado explícitamente como tal (el proceso *proc_estado*) que modela la memoria de estado y la lógica de cálculo del estado siguiente, y otro que modela la lógica de cálculo de la salida en función del estado.

```
library ieee;
use ieee.std_logic_1164.all;

entity det_011 is
 port(an_rst, clk : in std_logic;
      data_in     : in std_logic;
      det         : buffer std_logic
 );
end entity det_011;

architecture rtl of det_011 is
 type t_estado is (nada, d_0, d_01, d_011);
 signal estado : t_estado;
begin

 proc_estado : process(an_rst, clk)
 begin
  if an_rst = '0' then
    estado <= nada;
  elsif clk'event and clk = '1' then
    case estado is
      when nada =>
        if data_in = '0' then
          estado <= d_0;
        end if;
      when d_0 =>
        if data_in = '1' then
          estado <= d_01;
        end if;
```

```
      when d_01 =>
       if data_in = '0' then
        estado <= d_0;
       else
        estado <= d_011;
       end if;
      when d_011 =>
       if data_in = '0' then
        estado <= d_0;
       else
        estado <= nada;
       end if;
    end case;
  end if;
 end process proc_estado;

proc_salida : process(estado)
 begin
  if estado = d_011 then
   det <= '1';
  else
   det <= '0';
  end if;
 end process proc_salida;

end architecture rtl;-- of det_011
```

Ejemplo 3: sumador serie

Como tercer ejemplo usaremos un sumador serie que recibe los sumandos bit a bit por dos entradas empezando por el bit menos significativo, y entrega el resultado de la suma en serie por la salida, con el mismo número de bits que los sumandos (sumandos y resultado de N bits). Una entrada adicional indica, cuando se activa a nivel alto, que las entradas de datos presentes en ese ciclo de reloj contienen los bits de menos peso de un nuevo par de sumandos.

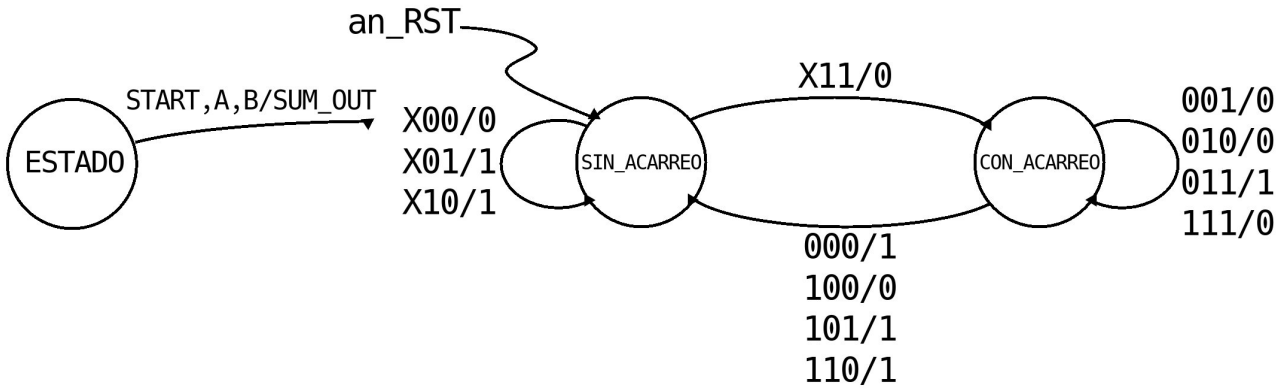
El hecho de que el resultado de la suma de cada pareja de bits deba presentarse en la salida en el mismo ciclo de reloj en que se presentan los datos de entrada obliga a que el diseño del autómata se realice según la arquitectura de Mealy. A continuación seguiremos los pasos de la metodología, al igual que en los ejemplos anteriores.

1. Identificación y codificación de entradas y salidas

Llamaremos *A* y *B* a las entradas de datos de un bit en las que se van presentando los operandos, y *START* a la entrada de un bit que indica a nivel alto el comienzo de un nuevo par de sumandos. Llamaremos *SUM_OUT* a la salida de un bit por la que se irá presentando el resultado de la suma.

2. Diseño del diagrama de estados

La siguiente figura muestra el diagrama de estados del sumador serie. Examínelo detenidamente hasta asegurarse de que comprende su funcionamiento.



3. Codificación del modelo VHDL

A continuación se muestra el modelo VHDL del autómata de Mealy que implementa el sumador serie. Fíjese en los dos procesos, uno sensible únicamente al *reset* asíncrono y al reloj que implementa la memoria de estado y la lógica de cálculo del estado siguiente y otro sensible al estado y a las entradas que implementa la lógica de cálculo de la salida.

```
library ieee;
use ieee.std_logic_1164.all;

entity serial_adder is
 port(an_rst, clk : in std_logic;
      a, b       : in std_logic;
      start      : in std_logic;
      sum_out    : buffer std_logic
 );
end entity serial_adder;

architecture rtl of serial_adder is
 type t_estado is (sin_acarreo, con_acarreo);
 signal estado : t_estado;
begin

 proc_estado : process(an_rst, clk)
 begin
  if an_rst = '0' then
    estado <= sin_acarreo;
  elsif clk'event and clk = '1' then
    case estado is
      when sin_acarreo =>
        if a = '1' and b = '1' then
          estado <= con_acarreo;
        end if;
      when con_acarreo =>
        if(start = '0' and a = '0' and b = '0') or
          (start = '1' and(a /= '1' or b /= '1') then
          estado <= sin_acarreo;
        end if;
    end case;
  end if;
 end process proc_estado;
```

```
  proc_out : process(estado, a, b, start)
  begin
    case estado is
      when sin_acarreo =>
       if a /= b then
         sum_out <= '1';
       else
         sum_out <= '0';
       end if;
      when con_acarreo =>
       if(start = '1' and a /= b) or (start = '0' and a = b) then
         sum_out <= '1';
       else
         sum_out <= '0';
       end if;
    end case;
  end process proc_out;

end architecture rtl;-- of serial_adder
```